

Mini Hiring Pipeline — Architecture & Design

1. Overview

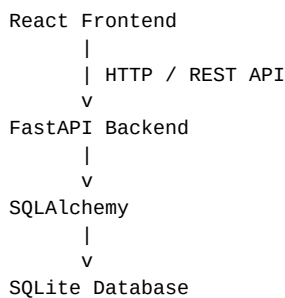
Mini Hiring Pipeline is a lightweight recruiter application for managing candidates through a fixed hiring workflow.

The pipeline is Applied → Screening → Interview → Offer → Hired.

Candidates can also be rejected at any stage before Hired.

The application provides candidate management, stage validation, audit history, current-stage duration, and recruiter search.

2. Architecture



3. Frontend

The frontend is implemented using React and Vite.

Responsibilities:

- Display candidates by pipeline stage
- Add candidates
- Move candidates to the next stage
- Reject candidates
- Display candidate details
- Display complete stage history
- Display time spent in the current stage
- Provide recruiter search
- Display search explanations and errors

Axios is used to communicate with the FastAPI backend.

4. Backend

The backend is implemented using Python and FastAPI.

Responsibilities:

- Candidate creation
- Candidate retrieval
- Stage transition validation
- Candidate rejection
- Stage history creation
- Candidate detail retrieval
- Search query processing

The backend enforces the hiring workflow rules.

5. Database Design

SQLite is used as the local database.

Candidate:

id, name, email, current_stage, created_at

StageHistory:

id, candidate_id, from_stage, to_stage, timestamp

A candidate can have multiple StageHistory records, allowing the complete hiring journey to be reconstructed.

6. Stage Transition Rules

The normal pipeline is:

```
Applied
↓
Screening
↓
Interview
↓
Offer
↓
Hired
```

Only one stage forward is allowed.

Applied → Interview is invalid because Screening was skipped.

A candidate can be rejected before reaching Hired.

Hired and Rejected are final states and cannot be changed afterward.

7. Audit Trail

Every stage change creates a new StageHistory record.

Example:

Applied

Applied → Screening

Screening → Interview

Interview → Offer

Offer → Hired

History records contain timestamps and are displayed in the candidate detail view.

8. Current Stage Duration

The current stage duration is calculated using the timestamp of the latest stage transition.

This supports both the candidate detail view and searches for candidates who have remained in a stage for more than one week.

9. Search Design

The application provides a single search box.

Supported queries include:

Find Priya Sharma

Find Priya Sharam

Who's in Interview right now?

Who has been stuck in Screening for more than a week?

Who moved to Interview since Monday?

Who reached the Offer stage but didn't get hired?

Everyone except rejected candidates.

The search implementation uses deterministic query parsing.

Name searches use similarity matching to tolerate common typing mistakes.

Unsupported queries return an explanation instead of silently returning an empty result.

10. API Design

```
GET /
GET /candidates
POST /candidates
POST /candidates/{candidate_id}/move
GET /candidates/{candidate_id}
GET /search?q={query}
```

11. Design Decisions

Backend-enforced workflow:

Stage validation is implemented on the backend so invalid transitions cannot be bypassed through the frontend.

SQLite:

SQLite keeps the application lightweight and local without requiring an external database server.

Separate history table:

Stage changes are stored separately instead of overwriting previous stages, providing an audit trail.

Deterministic search:

Required search cases use explicit query parsing rather than an external search engine or LLM dependency.

12. Error Handling

The API handles:

- Candidate not found
- Invalid stage
- Invalid stage transition
- Attempt to modify Hired candidate
- Attempt to modify Rejected candidate
- Empty search query
- Unsupported search query

The frontend displays backend search errors and explanations.

13. Future Improvements

- Automated backend tests
- More flexible combined search filters

- Authentication
- Recruiter/user roles
- Pagination
- Sorting and filtering
- Stronger database-level audit protection
- Production deployment
- Timezone-aware timestamps

14. Technology Stack

Frontend: React + Vite

API: FastAPI

Language: Python

ORM: SQLAlchemy

Validation: Pydantic

Database: SQLite

HTTP Client: Axios

15. Summary

The system separates the presentation layer, API/business logic, and persistence layer.

The backend owns the hiring workflow rules and maintains the candidate audit trail, while the React frontend provides the recruiter interface.

The design keeps the application simple, local, deterministic, and easy to run.